IA para Desarrolladores

Sesión 3 · Hacer — contexto, reglas del equipo y especificación

CURSO · 20 HORAS · 10 SESIONES

Agenda de hoy

10'	20'	15'	15'	15'	30'	15'
Repaso y aprobación de intenciones	Prompts y contexto: gastar menos tokens	Demo: reglas por tipo de archivo	Las herramientas del banco por MCP	Demo: /spec	Práctica: el spec.md de mi reto	Preguntas, tarea y cierre

Hoy practican de nuevo en su máquina, sobre su mi-reto con el intent.md aprobado.

Dónde vamos en el ciclo

1 · Intención	2 · Especificación	3 · Plan	4 · Construcción	5 · Revisión	6 · Entrega
`intent.md`	`spec.md`	`plan.md`	`código y pruebas`	`revision.md`	`CAMB-####`
Sesión 2 ✓	**Hoy**	Sesión 4	S4–S6	S5–S7	S8

Lo que hoy se suma al agente del equipo (versión 1)

Reglas por tipo de archivo

Pruebas, APIs, ETL y artefactos: se cargan solo al tocar esos archivos.

Servidor MCP del banco

glosario, consultar_politica y catalogo_servicios, en las dos herramientas.

Skill /spec

De la intención aprobada a la especificación, consultando al banco.

Antes de especificar: ¿la intención está aprobada?

- La métrica de éxito se puede medir: hoy y meta
- Los datos están clasificados con banco.yaml y son sintéticos
- Las reglas citan su clave; los valores inventados están en Supuestos
- Lo que queda fuera está escrito en "No incluye"
- Las preguntas abiertas están anotadas

Una o dos personas comparten su intent.md

```
| Estado       | aprobado |
| Aprobado por | <nombre> |
```

Sin esto, /spec se detiene: el ciclo no salta etapas.

Prompts y contexto

Cómo pedir bien y cómo gastar menos tokens

Un buen prompt tiene cinco partes

1

Objetivo

Qué quiere, en una frase con verbo: escribe, explica, corrige.

2

Contexto

Qué archivos o datos: @archivo en Claude Code, #archivo en Copilot.

3

Restricciones

Qué no debe hacer: no cambiar src/, solo Decimal, sin dependencias nuevas.

4

Formato

Cómo quiere la salida: un archivo, una tabla, tres líneas.

5

Terminado

Cómo sabe que acabó: las pruebas pasan, responde cada pregunta.

Lo que siempre aplica no va en el prompt: va en AGENTS.md, en una regla o en una skill. El prompt dice lo de esta tarea.

El mismo pedido, dos veces

Malo

```
haz pruebas para el gmf
```

- ¿Qué archivo? ¿Dónde van las pruebas?
- ¿Qué reglas importan?
- ¿Puede cambiar el código?
- ¿Cuándo terminó?

Bueno

```
Escribe las pruebas de src/pagos/gmf.py
en tests/test_gmf.py.          ← objetivo
Cubre las reglas de banco.yaml
(GMF, mínimo, límite) y bordes. ← contexto
No cambies src/; usa Decimal.  ← restricciones
Dime qué regla cubre cada una. ← formato
Termina cuando pytest pase.    ← terminado
```

Un prompt vago no ahorra tokens: el agente explora de más, adivina y hay que corregir. La corrección cuesta más que el prompt.

Qué llena la ventana de contexto

← lo que ya trae antes de que usted escriba *lo que agrega la tarea →*



Instrucciones de la herramienta	AGENTS.md · CLAUDE.md	Descripciones de skills y herramientas MCP	Reglas por tipo de archivo	Archivos que lee y salidas de comandos	La conversación
siempre	siempre	siempre (el cuerpo de la skill, solo al usarla)	al tocar esos archivos	lo que más crece	cada mensaje, de ida y vuelta

La ventana tiene un límite y todo cuesta tokens. Cuando se llena, el agente olvida lo del principio o la herramienta resume la conversación.

Cómo gastar menos tokens

Hábito	Claude Code	Copilot Chat
Una conversación por tarea	/clear	+ (conversación nueva)
Resumir una conversación larga	/compact	conversación nueva con un resumen
Apuntar a archivos concretos	@src/pagos/gmf.py	#file o arrastrar el archivo
No pegar logs enteros	pida el resumen del error	pida el resumen del error
Modelo según la tarea	/model (haiku para lo rápido)	selector de modelo (Haiku para lo rápido)
Ver qué ocupa la ventana	/context	indicador de la ventana en la barra del chat
Instrucciones cortas, lo demás bajo demanda	CLAUDE.md corto · skills · reglas	AGENTS.md corto · skills · instrucciones

En el agente: menos contexto, el correcto

AGENTS.md · nueva sección

```
## Contexto y tokens
- Lea solo los archivos que la tarea
  necesita; no recorra carpetas enteras.
- Si una salida es larga, resuma lo
  importante en lugar de repetirla.
- Para una tarea nueva, conversación
  nueva.
```

CLAUDE.md · ya no importa banco.yaml

```
@AGENTS.md

- Las definiciones del banco están en
  banco.yaml. No se importa aquí para no
  cargarlo en cada conversación: léalo
  cuando la tarea lo necesite, y para el
  glosario, las políticas y el catálogo
  use el servidor MCP banco.
```

banco.yaml creció (glosario, políticas, catálogo): ahora el agente pregunta solo lo que necesita, con las herramientas del banco.

Reglas por tipo de archivo

Un error que se repite se vuelve regla

Dos clases de reglas

`AGENTS.md`
Lo que siempre aplica

Stack, nombres, dinero, datos, el ciclo. Se lee en cada conversación: corto.

`.claude/rules · .github/instructions`
Lo que aplica a ciertos archivos

Pruebas, APIs, ETL, artefactos. Se carga solo cuando el agente toca esos archivos.

Regla del equipo (v1)	Aplica a	Por ejemplo
pruebas	tests/**/*.py	cada prueba cita su regla de banco.yaml; bordes con parametrize
api	**/api/**/*.py	rutas /v1 en plural, errores {codigo, mensaje, id_correlacion}
etl	**/etl/**/*.py	umbrales desde configuración, conteo de registros rechazados
artefactos	docs/sdlc/**/*.md	el agente deja borrador; valores sin clave van a Supuestos

Una regla, dos formatos

Claude Code · .claude/rules/pruebas.md

```
---
description: Cómo escribimos las pruebas
paths:
  - "tests/**/*.py"
  - "**/test_*.py"
---

# Reglas para las pruebas
- Cada prueba empieza con un docstring
  que cita su regla de banco.yaml
- Bordes con pytest.mark.parametrize
```

Copilot · .github/instructions/pruebas.instructions.md

```
---
description: "Cómo escribimos las pruebas"
applyTo: "tests/**/*.py,**/test_*.py"
---
<!-- Generado desde .claude/rules/... -->

# Reglas para las pruebas
- Cada prueba empieza con un docstring
  que cita su regla de banco.yaml
- Bordes con pytest.mark.parametrize
```

Se escribe una vez en .claude/rules y python herramientas/sincronizar_skills.py la lleva a Copilot: paths se vuelve applyTo.

Demo: las mismas pruebas, antes y después

Sin la regla

```
def test_validar_monto_acepta_monto_minimo():
    validar_monto(Decimal("1000"))

def test_validar_monto_rechaza_bajo_el_minimo():
    with pytest.raises(MontoInvalido):
        validar_monto(Decimal("999"))
```

Con la regla

```
@pytest.mark.parametrize("monto, esperado", [
    (Decimal("999.99"), False),
    (Decimal("1000"), True),
    (Decimal("1001"), True)],
    ids=["justo_por_debajo", "justo_en_el_minimo",
         "justo_por_encima"])
def test_monto_minimo_bordes(monto, esperado):
    """Regla: productos.transferencias.
       monto_minimo."""
```

Las dos pasan. La diferencia es que la segunda dice qué regla del banco verifica y prueba los bordes juntos.

No hubo que pedirlo en el prompt: la regla se cargó al escribir en tests/.

Cómo se escribe una buena regla

Concreta

Sí: "Cada prueba cita su clave de banco.yaml"

No: "Escriba buenas pruebas"

Verificable

Sí: Se puede revisar en el diff: está o no está

No: "Sea claro y ordenado"

Con ejemplo

Sí: """Regla: productos.transferencias.gmf."""

No: Sin ejemplo, cada agente lo entiende distinto

Acotada

Sí: paths: tests/**/*.py

No: Todo en AGENTS.md, para todos los archivos

Su tarea: una regla propia, a partir de algo que su asistente hizo mal o distinto dos veces.

Las herramientas del banco por MCP

El agente consulta al banco en lugar de adivinar

¿Qué es MCP?



MCP (Model Context Protocol) es un estándar para que un agente use herramientas de otros sistemas: el servidor dice qué herramientas tiene y el agente las llama.

El mismo servidor sirve a las dos herramientas. Es un programa en Python que corre en su máquina y no sale a internet.

En el banco real el servidor consultaría sus sistemas; el agente no cambia.

Las tres herramientas del banco (v0)

`glosario`

Qué significa un término de negocio

```
glosario("usura")
```

```
usura: Tasa máxima que la
ley permite cobrar. En el
banco ficticio es
productos.credito_consumo
  .tasa_maxima_ea
```

`consultar_politica`

La política sobre datos, seguridad, nombres, logs, apis o pruebas

```
consultar_politica("logs")
```

```
Política de logs:
JSON con fecha, nivel,
servicio, id_correlacion;
nunca datos restringidos
```

`catalogo_servicios`

Qué APIs internas ya existen y qué devuelven

```
catalogo_servicios("tasas")
```

```
svc-tasas
GET /v1/tasas/{producto}
devuelve:
  tasa_ea_vigente [publica]
  tasa_maxima_ea [publica]
```

Conectarlo en las dos herramientas

Claude Code · .mcp.json

```
{
  "mcpServers": {
    "banco": {
      "type": "stdio",
      "command": "<Python del curso>",
      "args": ["herramientas/mcp_banco/servidor.py"]
    }
  }
}
```

Copilot · .vscode/mcp.json

```
{
  "servers": {
    "banco": {
      "type": "stdio",
      "command": "<Python del curso>",
      "args": ["${workspaceFolder}/herramientas/
               mcp_banco/servidor.py"]
    }
  }
}
```

python herramientas/configurar_mcp.py escribe los dos archivos con el Python de su entorno (el que tiene PyYAML).

Claude Code: abra claude de nuevo, acepte el servidor y revise con /mcp. **Copilot:** Start sobre banco en .vscode/mcp.json.

Llevar su repositorio a la versión 1

```
$ python proyectos/actualizar_repositorio.py ../mi-reto

Agregados (16):     .claude/rules/..., .claude/skills/spec/,
                    herramientas/mcp_banco/servidor.py, .mcp.json ...
Actualizados (6):   banco.yaml, CLAUDE.md, README.md ...
Usted los modificó, no se tocaron (1):
  ! AGENTS.md  ->  AGENTS.md.nuevo
Servidor MCP configurado con este Python.
Commit hecho.
```

Agrega lo que no tiene.

Actualiza lo que usted no tocó.

No pisa lo que usted cambió: deja el .nuevo al lado.

Nunca toca docs/sdlc: sus artefactos.

Así se actualizará el agente en el banco: cada proyecto recibe la nueva versión de la plantilla sin perder lo suyo.

De la intención a la especificación

La skill /spec

Intención, especificación y plan

	intent.md	spec.md	plan.md
Responde	¿Qué problema y para quién?	¿Qué debe hacer el sistema?	¿Cómo lo construimos?
Lo aprueba	Dueño del producto	Dueño del producto	Líder técnico
Trae	Métrica, alcance, datos, reglas	Requisitos con criterios de aceptación, reglas, datos, servicios, políticas	Componentes, capas, pasos, pruebas
No trae	Soluciones técnicas	Código ni diseño de clases	El código
Skill	/intencion · S2	/spec · hoy	/plan · S4

La especificación es el contrato: de cada criterio de aceptación sale una prueba en la sesión 4.

Qué hace la skill /spec

1

¿Aprobada?

Lee intent.md. Si no dice aprobado, se detiene.

2

Consulta al banco

glosario, consultar_politica y catalogo_servicios por MCP.

3

Pregunta

Máximo 3 preguntas, solo si no alcanza para los criterios.

4

Escribe spec.md

Con la plantilla: RF con Dado / Cuando / Entonces, RN con su clave.

5

Lo deja en borrador

Resumen, servicios usados y preguntas abiertas. Aprueba una persona.

Al escribir en docs/sdlc se carga además la regla de artefactos: borrador, claves de banco.yaml, nada inventado.

Así se ve un requisito en el spec.md

```
### RF-04 · Verificar capacidad de pago
El sistema debe calcular el endeudamiento
(cuota + otras cuotas) / ingresos y dar un veredicto.

- Dado ingresos 5.000.000, otras cuotas 500.000
  y una cuota de 1.500.000 cuando se simula
  entonces endeudamiento = 0,40 y "alcanza".
- Dado una cuota de 1.500.001 ... entonces
  "no alcanza".
```

Id	Regla	Fuente
RN-01	Tasa máxima (usura)	`productos.credito_consumo.tasa_maxima_ea`
RN-05	Endeudamiento máximo	`productos.credito_consumo.endeudamiento_maximo`

Y además: datos clasificados, servicios del catálogo (svc-tasas), políticas, casos borde, trazabilidad y preguntas abiertas.

Si no sabe un valor, no lo inventa: lo deja como pregunta abierta o lo calculamos aparte, nunca con el código que se va a probar.

¿Cuándo se aprueba un spec.md?

- Cada RF tiene criterios Dado / Cuando / Entonces con valores concretos y sintéticos
- Cada RN cita su clave de banco.yaml o su política; no hay valores inventados
- Cada dato tiene clasificación y origen
- Dice qué servicios del catálogo usa, o por qué ninguno
- La trazabilidad cubre las métricas de la intención
- Lo que la intención dejó fuera sigue fuera; no hay código

Aprobar = dos cambios y un commit

```
Estado: aprobado
Aprobado por: <nombre>

git add docs/sdlc
git commit -m
  "Especificación aprobada"
```

Manos a la obra: la especificación de mi reto

5'

Actualizar mi-reto

Desde la carpeta del curso:
actualizar_repositorio.py ../mi-reto
y verificar_equipo.py

5'

Conectar el MCP

Claude Code: claude, aceptar y /mcp.
Copilot: Start en .vscode/mcp.json.
¿Qué es la usura según el banco?

15'

Escribir el spec.md

/spec <caso> con su intent.md aprobado; responder sus preguntas

5'

Revisar juntos

Una o dos personas comparten pantalla con la lista de aprobación

¿Algo no funciona? verificar_equipo.py dice qué falta. Si /mcp no muestra banco, salga y vuelva a abrir claude.

Preguntas del grupo: ¿el agente tiene caché?

Caché del prompt

Automático

Qué es: el modelo reutiliza la parte fija del contexto (instrucciones, CLAUDE.md o AGENTS.md, herramientas): cuesta menos y responde más rápido.

Claude Code: activo por defecto; dura unos minutos sin uso.

Copilot: lo maneja GitHub por dentro.

Lo que depende de ustedes: que esa parte fija sea corta y no cambie a cada rato.

Caché de respuestas (FAQ)

No existe: se guarda en el lugar correcto

Respuesta corta que siempre aplica: una línea en AGENTS.md.

Un procedimiento: una skill (se carga al usarla).

Un dato del negocio: una herramienta MCP, como glosario o catalogo_servicios.

Muchas preguntas: docs/faq.md y una regla que diga cuándo leerlo.

El servidor MCP del banco es la FAQ del banco: la respuesta no ocupa la ventana hasta que alguien pregunta.

Preguntas del grupo: pruebas, ¿skill o subagente?

Pieza	Qué aporta a las pruebas	En el agente
Regla por tipo de archivo	Cómo se escriben: docstring con la regla, parametrize, Decimal	Ya está: .claude/rules/pruebas.md
Skill /pruebas	El procedimiento: qué probar, bordes, datos sintéticos	Sesión 6
Subagente Verificador	Corre las pruebas y reporta sin corregir, con un modelo rápido	Sesión 6
Hook	Las corre siempre, después de cada edición	Sesión 6

Skill = cómo: el agente principal sigue un procedimiento en la misma conversación.

Subagente = quién: su propio contexto, permisos y modelo. Úselo si la tarea genera mucha salida (logs de pytest), si necesita otros permisos o si conviene un modelo más barato.

Escribir pruebas es una skill; correrlas y juzgarlas, un subagente; que se corran siempre, un hook.

Preguntas del grupo: ¿importan las mayúsculas?

Archivo	Nombre exacto	Lo busca
CLAUDE.md	en mayúsculas	Claude Code
AGENTS.md	en mayúsculas	Copilot y otros agentes (estándar abierto)
SKILL.md	singular: SKILLS.md no se carga	las dos, dentro de la carpeta de cada skill
copilot-instructions.md · *.instructions.md · *.agent.md · .mcp.json	en minúsculas	Copilot y Claude Code

Windows y Mac no suelen distinguir mayúsculas: claude.md puede funcionar en su portátil.

Linux, Git y la nube (GitHub Actions, el agente de Copilot en GitHub) sí: ahí deja de funcionar. Para corregirlo en Windows: git mv claude.md CLAUDE.md.

¿Por qué mayúsculas? Es la tradición de README y LICENSE: archivos para leer primero, que salen arriba en los listados.

Regla práctica: siempre el nombre exacto, con las mismas mayúsculas, aunque en su máquina funcione de otra forma.

Para casa

1

Mi spec.md

Terminarlo con la lista, aprobarlo y hacer commit.

2

Una regla propia

Algo que su asistente hizo mal dos veces. En .claude/rules y sincronizar.

3

Entregar

spec.md y la regla por el canal, antes de la sesión 4.

4

Leer

Claude Code 101: contexto y CLAUDE.md. GH-600: diseño del agente e integración al ciclo.

Todo el detalle en sesion-03/TAREA.md.

Para la sesión 4

1

Del plan al código

La plantilla del proyecto por capas: dominio, casos de uso, puertos, adaptadores, app.

2

Dos agentes

El Arquitecto planea sin tocar código; el Desarrollador implementa con la prueba primero.

3

Lo que traen

Su spec.md aprobado y su regla propia.

Menos contexto, el correcto: el agente pregunta al banco en lugar de adivinar.